DD2424 — Assignment 1 Report

One-Layer Softmax Classifier on CIFAR-10

1 Introduction

This report describes the implementation and evaluation of a single-layer softmax classifier trained on the CIFAR-10 dataset using mini-batch gradient descent.

2 Gradient Verification

The gradients from `BackwardPass` were checked against PyTorch's autograd (`ComputeGradsWithTorch`) on a small problem ($d = 10$, $n = 3$) with $\lambda = 0$ and $\lambda = 0.1$. Results are summarized below

Table 1: Gradient verification results. All errors are at machine precision level.

	Max Abs Error (W)	Max Abs Error (b)	Max Rel Error (W)	Max Rel Error (b)
$\lambda = 0$	1.11e−16	5.55e−17	2.10e−15	1.19e−16
$\lambda = 0.1$	1.11e−16	5.55e−17	4.58e−15	1.19e−16

3 Training Results

The network was trained on four hyperparameter settings, all with `n_batch`=100 and `n_epochs`=40. Training used `data_batch_1` (10,000 images), validation was `data_batch_2`, and testing was on `test_batch`. Each dimension was normalized using training set statistics, and weights were drawn from a Gaussian ($\sigma = 0.01$, seed=42).

Table 2: Final accuracies after 40 epochs for each configuration.

Config	λ	η	Train Acc	Val Acc	Test Acc
1	0.0	0.1	42.12%	27.10%	27.70%
2	0.0	0.001	45.57%	38.46%	39.21%
3	0.1	0.001	44.63%	38.62%	39.30%
4	1.0	0.001	39.85%	36.32%	37.55%

3.1 Configuration 1: $\lambda = 0$, $\eta = 0.1$

A learning rate of 0.1 makes training very unstable, the loss swings between 3 and 10 across epochs and never really settles down. There's also a big gap between training accuracy (42%) and validation accuracy (27%), suggesting overfitting on top of the instability. The weight templates look like noise, so the network hasn't picked up on any meaningful patterns. Test accuracy lands at 27.70%, barely above the 10% random baseline, which confirms that this learning rate is just too high.

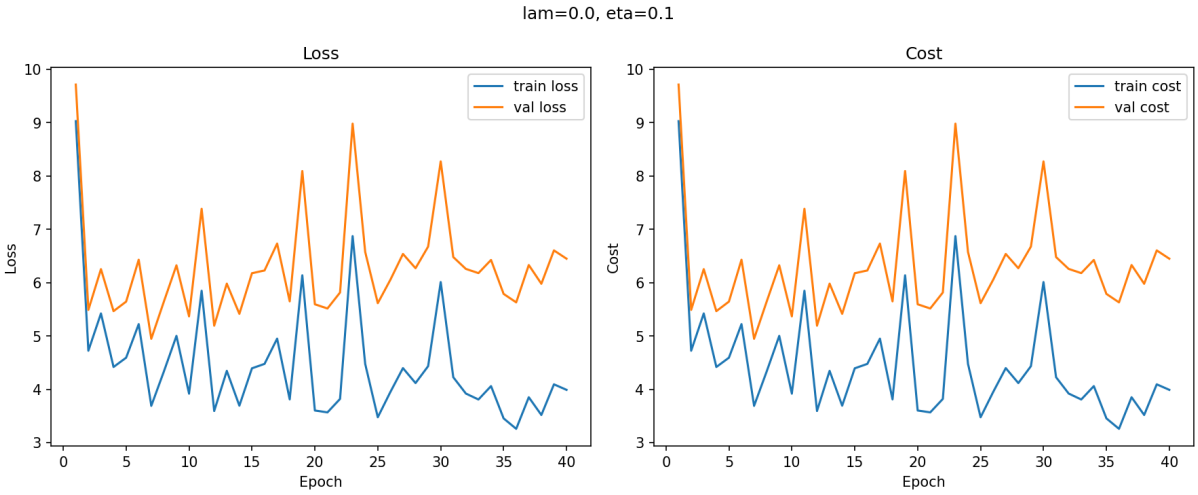

Figure 1: Loss and cost curves for $\lambda = 0$, $\eta = 0.1$. Highly unstable training.

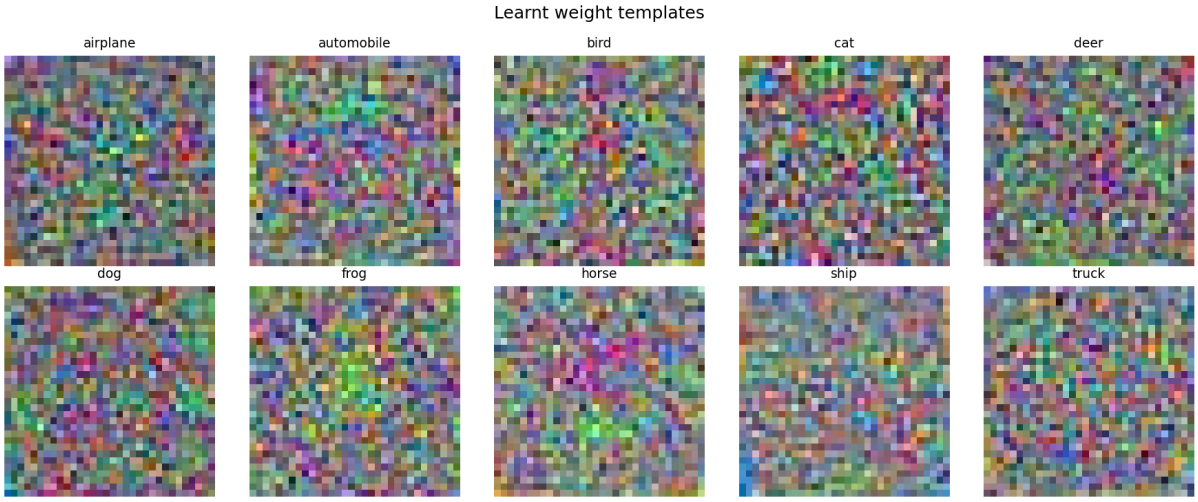

Figure 2: Weight templates for $\lambda = 0$, $\eta = 0.1$. No meaningful patterns learned.

3.2 Configuration 2: $\lambda = 0$, $\eta = 0.001$

Dropping the learning rate to 0.001 gives smooth, steadily decreasing loss curves. Training loss gets down to 1.61 while validation loss flattens out around 1.79, a growing gap that's been expected with no regularization. Since $\lambda = 0$, the loss and cost curves are the same. The weight templates still look a little noisy, since nothing stops the weights from taking on large, high-frequency values. Test accuracy comes in at 39.21%, close to the reference geiven in the assignment guidelines, i.e. 39.13%.

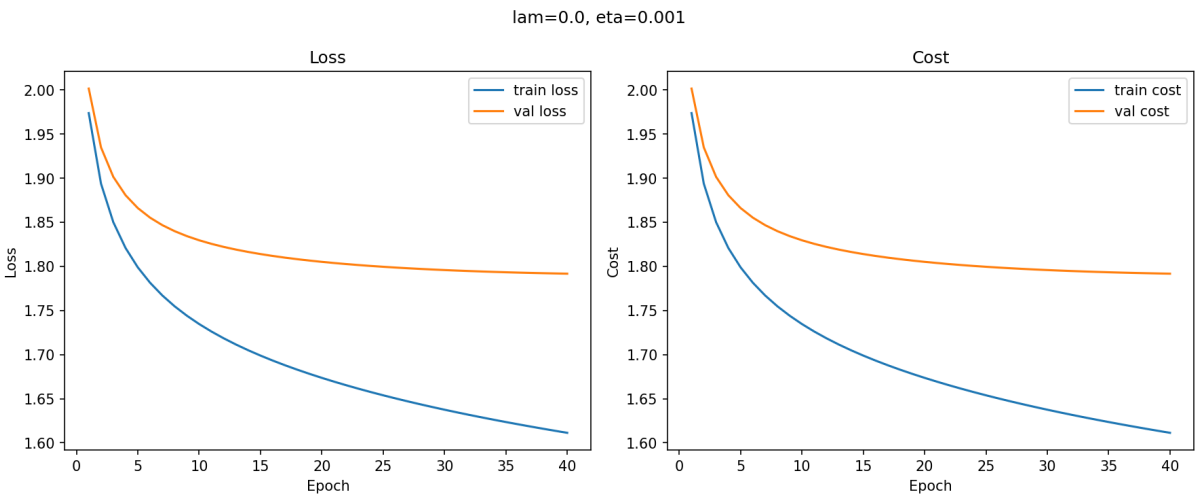

Figure 3: Loss and cost curves for $\lambda = 0$, $\eta = 0.001$. Smooth convergence, some overfitting.

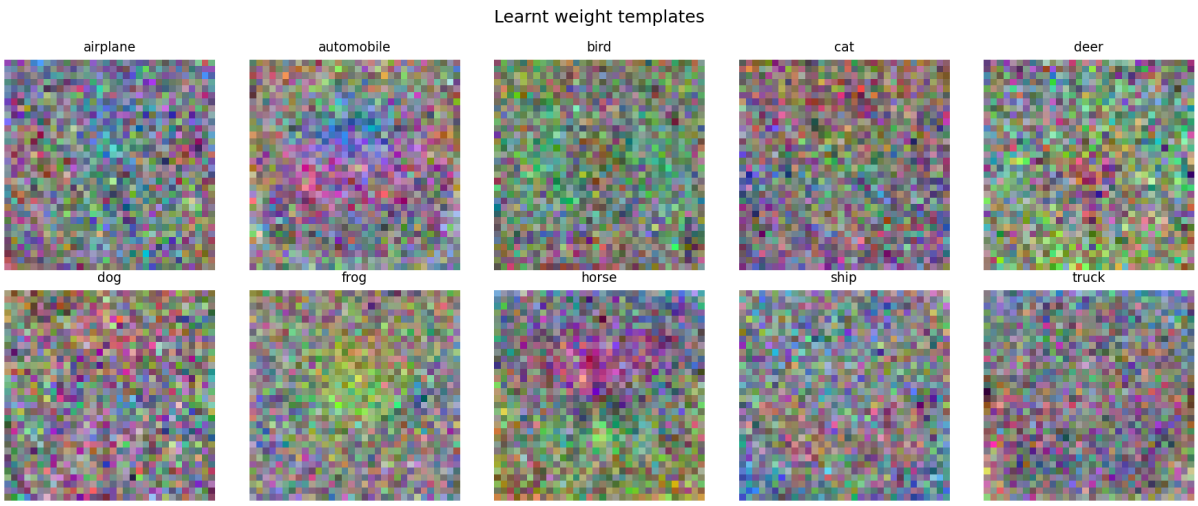

Figure 4: Weight templates for $\lambda = 0$, $\eta = 0.001$. Noisy templates due to no regularization.

3.3 Configuration 3: $\lambda = 0.1$, $\eta = 0.001$

Adding a little moderate regularization ($\lambda = 0.1$) gives loss curves that look a lot like Config 2, but the cost plot tells a different story, the regularization penalty opens up a gap between cost and loss, and the train/val cost curves sit closer together than without regularization. The loss curves themselves (which don't include the penalty) show a slightly smaller train-validation gap than Config 2, so regularization is helping with overfitting. The weight templates start showing slightly more better structure, and test accuracy goes up to 39.30%.

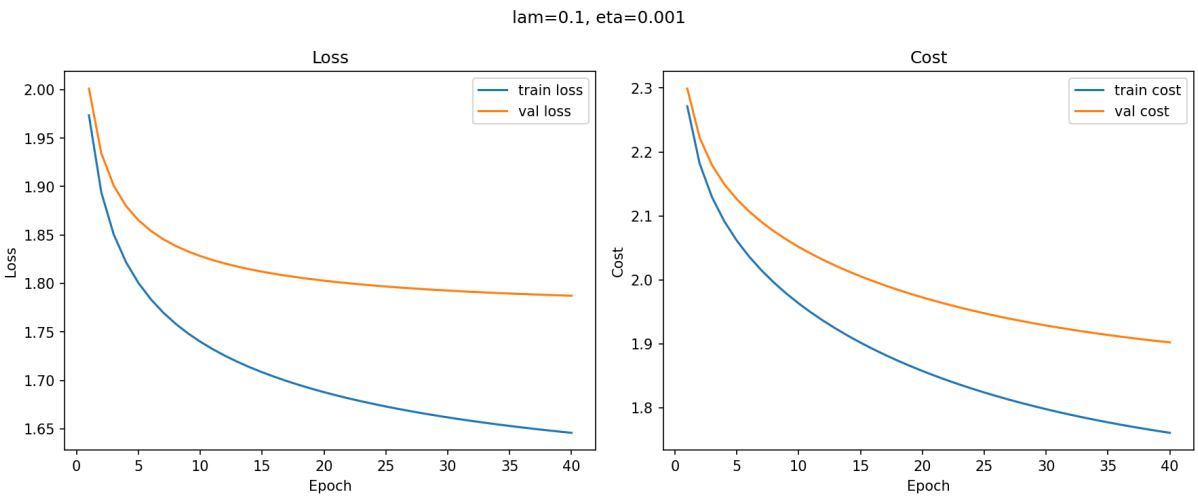

Figure 5: Loss and cost curves for $\lambda = 0.1$, $\eta = 0.001$. Cost includes the regularization term.

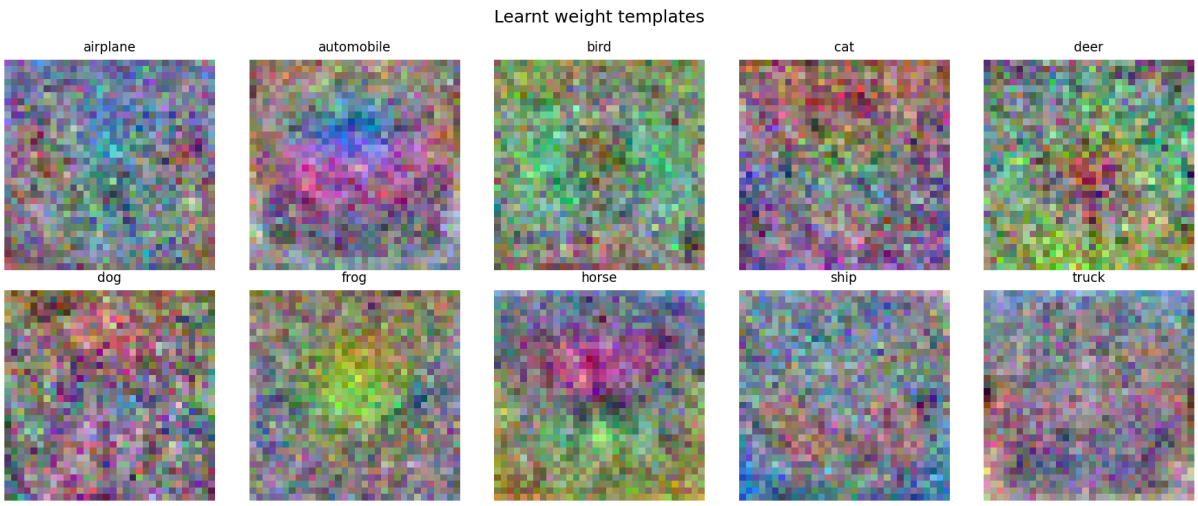

Figure 6: Weight templates for $\lambda = 0.1$, $\eta = 0.001$. Slightly more structured patterns emerging.

3.4 Configuration 4: $\lambda = 1.0$, $\eta = 0.001$

Strong regularization ($\lambda = 1.0$) is where things get interesting. Train and validation loss nearly overlap, so overfitting is basically gone, but the loss flatens early (around epoch 15) at $\sim$1.80, higher than the $\sim$1.61 in Configs 2 and 3, meaning the network underfits. The cost starts high ($\sim$4.0) due to the penalty on random initial weights, then drops fast as they shrink. The weight templates are now actually readable: airplane picks up blue sky, frog is green, ship has blue water, horse shows a brown body. Test accuracy falls to 37.55%.

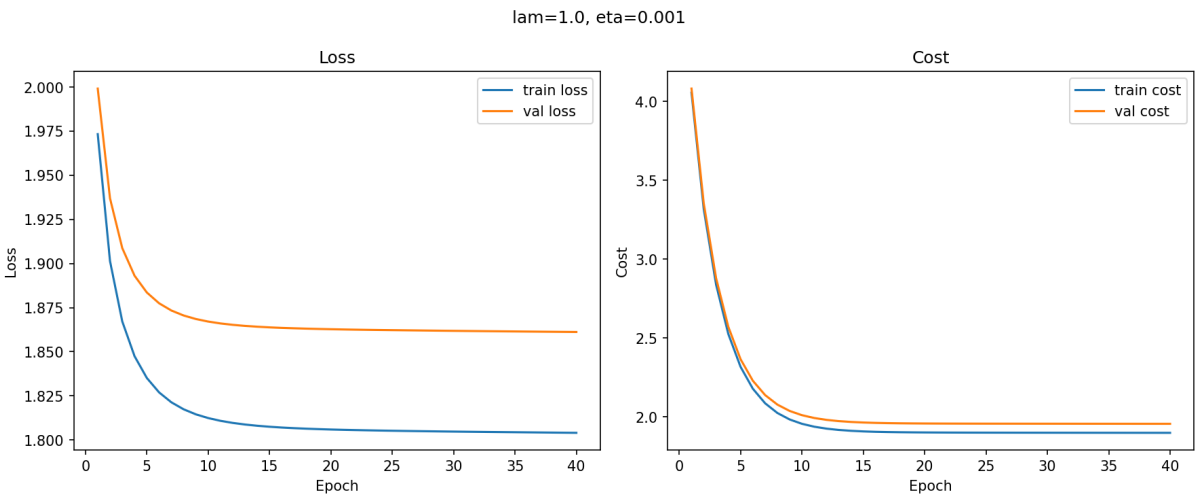

Figure 7: Loss and cost curves for $\lambda = 1.0$, $\eta = 0.001$. Minimal overfitting but early plateau.

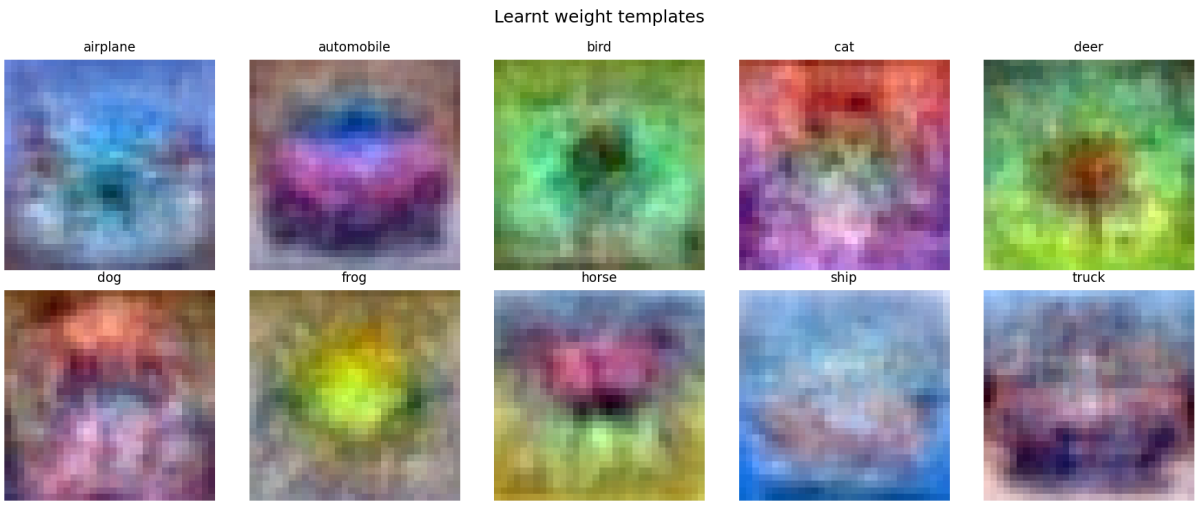

Figure 8: Weight templates for $\lambda = 1.0$, $\eta = 0.001$. Highly interpretable class templates.

4 Exercise 2.1: Bonus Improvements

Four improvements were applied cumulatively on top of the baseline (Config 3):

1. **All training data** : 49k images instead of 10k (1k held out for validation).
2. **Horizontal flip augmentation** : each image flipped with p=0.5 per mini-batch.
3. **Step-decay LR** : $\eta \times 0.1$ every 20 epochs.
4. **Grid search** : 3×3×3 over $\eta \in \{0.01, 0.005, 0.001\}$, $\lambda \in \{0.01, 0.05, 0.1\}$, batch $\in \{50, 100, 200\}$.

The biggest gain came from simply using more training data. With augmentation in the mix, the network didn't need as much L2 regularization, so λ=0.01 consistently outperformed λ=0.1. The best setup ended up being η=0.005, λ=0.01, batch size 200, hitting **42.80% validation** and **42.30% test accuracy**, up from 39.30% at baseline.

Table 3: Top grid search results (val acc, 40 epochs, step decay, flip aug, 49k train).

η	λ	`n_batch`	Val Acc
0.005	0.01	200	**42.80%**
0.001	0.01	50	42.70%
0.001	0.01	100	42.10%
0.001	0.01	200	42.40%
0.005	0.01	50	41.80%

5 Exercise 2.2: Multiple Binary Cross-Entropy Loss

Gradient

Replacing softmax with element-wise sigmoid and using the multiple BCE loss $l_{\text{BCE}} = -\frac{1}{K}\sum_k[(1-y_k)\log(1-p_k) + y_k \log(p_k)]$, the gradient w.r.t. scores is:

$$\frac{\partial l_{\text{BCE}}}{\partial s_k} = \frac{1}{K}(p_k - y_k)$$

identical in form to the softmax case but scaled by $1/K$.

Results

Both models trained with batch= 50, λ=0.01, step decay, flip aug, 49k images. BCE used η=0.01 (scaled by K) vs. η=0.001 for softmax.

Model	η	Test Acc
Softmax CE	0.001	42.22%
Sigmoid BCE	0.01	41.13%

Histogram comparison

Softmax concentrates correct-class probabilities higher (due to the $\sum p_k$=1 constraint), while BCE spreads them more evenly since classes don't compete. The BCE model shows less overconfidence on misclassified examples. Both models show similar overfitting in their loss curves.

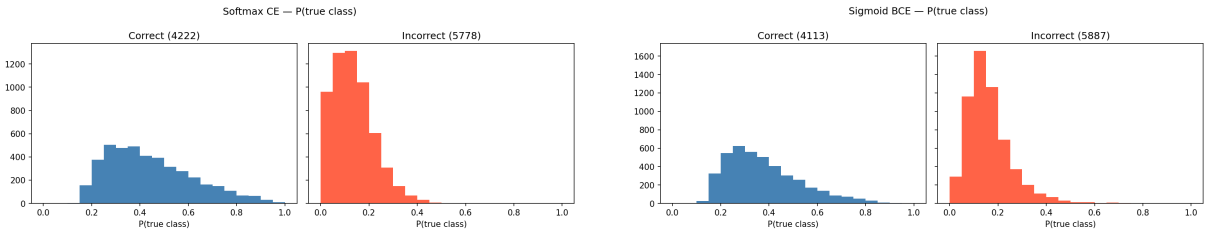

Figure 9: P(true class) for correct (blue) vs. incorrect (red) test examples. Left: softmax. Right: sigmoid BCE.